Instituto Politécnico Nacional

UPIITA (Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas)

Base de Datos Distribuidos

Profesor: De la cruz Sosa Carlos

Asignación: Consultas

Alumno: -González Luna Marco

-Gutiérrez Flores José Julián

-Muñoz Reyes Citlali

-Olvera Valdivia Cristóbal

Consulta 1 - Gutiérrez Flores José Julián

Consulta Original:

```
SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate, c.Name AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE YEAR(soh.OrderDate) = 2014 AND p.ListPrice > 1000;
```

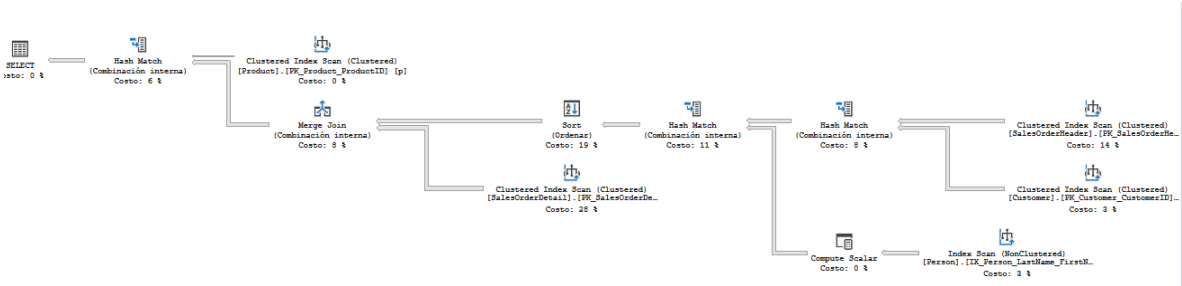
Consulta modificada para que funcione con el elemento Name:

```
SELECT
    p.Name AS Producto,
    sod.OrderQty,
    soh.OrderDate,
    per.FirstName + ' ' + per.LastName AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
WHERE YEAR(soh.OrderDate) = 2014 AND p.ListPrice > 1000;
```

- ***Análisis de la consulta original:***

La consulta tiene como objetivo generar un reporte detallado que asocie el nombre de los productos con sus cantidades vendidas, fechas de orden y el nombre completo del cliente. Para lograr esto, realiza una cadena de cinco uniones (JOINs) que conectan el catálogo de productos con el historial de ventas y, finalmente, con los datos personales de los clientes en el esquema Person.

- ***Plan de ejecución de consulta original***:



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	2026-03-26	10281	79	2167	50%	Predicado No SARGable (YEAR) y alto I/O	Cambiar a rango de fechas y optimizar índices en SalesOrderDetail

Ya con la modificación a la consulta proporcionada por el profesor para que funcione en nuestra base de datos (AdventureWorks2022). Activamos los parámetros de SET STATISTICS, para observar el rendimiento real de la consulta.

La consulta base ocupa un YEAR(OrderDate) = 2014, que obliga a SQL a calcular el año a cada una de las filas para después seleccionar las que cumplían la condición, por lo que en primera instancia se agrego un rango de fechas, lo que hizo que se usara un Index Seek, y obliga a saltar directo al rango de nuestras fechas.
>= '20140101' AND < '20150101'
Otro punto fue que el p.ListPrice > 1000 filtraba después de que SQL uniera todas las ventas con los productos para aplicar el filtro, por lo que se creo una subconsulta, lo que género que SQL filtrara primero los productos caros y para después buscar las ventas. Esto redujo el volumen de datos que "viajan" por los JOINs.
(SELECT ... FROM Product WHERE ListPrice > 1000) p
Además, se hizo la creación de índices que permitieran la reducción de las consultas lógicas. La consulta original presentaba un alto costo de I/O (Entrada/Salida) debido a que el motor de SQL Server se veía obligado a realizar escaneos completos de tablas (Clustered Index Scans). Para mejorar el rendimiento de la consulta, se implementaron dos índices no agrupados (Non-Clustered Indexes).
El primer índice se hizo en Sales.SalesOrderDetail, la tabla era de las más voluminosas. Ya que originalmente, SQL realizaba un escaneo de 1,238 lecturas lógicas. Al definir ProductID como la llave del índice, permitimos que el motor realice un Index Seek (búsqueda directa) para unir los productos filtrados por precio. Y al incluir OrderQty y SalesOrderID dentro del índice, evitamos el Key Lookup. Esto significa que SQL encontró toda la información necesaria dentro del mismo índice, reduciendo las lecturas a 303 (una mejora del 75%).

```
CREATE INDEX IX_SalesOrderDetail_ProductID_Optimization
ON Sales.SalesOrderDetail (ProductID)
INCLUDE (SalesOrderID, OrderQty);
```

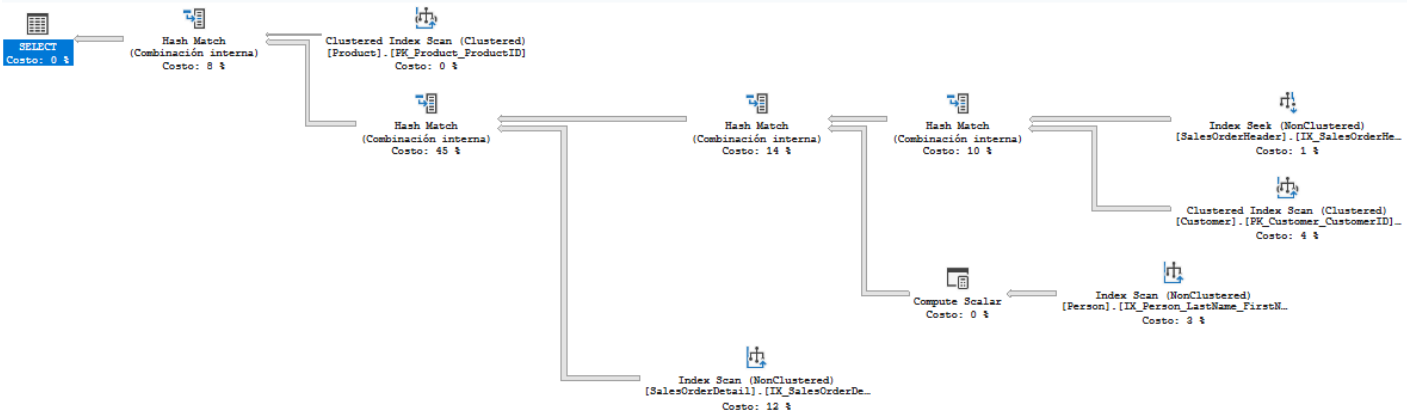
El segundo índice se hizo en Sales.SalesOrderHeader, el filtro principal de la consulta es el año (OrderDate). Sin este índice, SQL leía todas las cabeceras de pedidos de todos los años disponibles. Por lo que al colocar OrderDate como columna de búsqueda, el motor pudo saltar directamente al rango de fechas de 2014. Esto desplomó las lecturas lógicas de 686 a solo 35 (una reducción de eficiencia del 95%).

```
CREATE INDEX IX_SalesOrderHeader_DateCustomer_Optimization
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (CustomerID, SalesOrderID);
```

- ***Propuesta de solución***

```sql
SELECT
    p.Name AS Producto,
    p.ListPrice,
    sod.OrderQty,
    soh.OrderDate,
    per.FirstName + ' ' + per.LastName AS Cliente
FROM (
    SELECT ProductID, Name, ListPrice
    FROM Production.Product
    WHERE ListPrice > 1000
) p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
WHERE soh.OrderDate >= '20140101'
  AND soh.OrderDate < '20150101';
```

❖ ***Plan de ejecución real:***

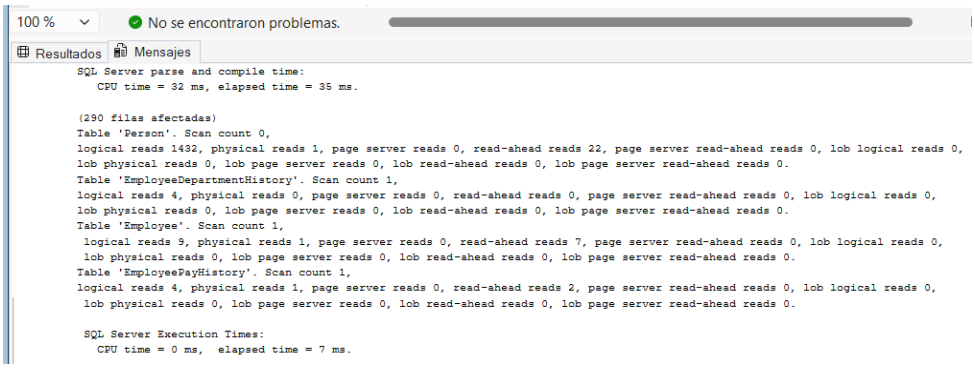


Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	2026-03-28	251	78	581	50%	Ninguno significativo	Mantener el uso de rangos de fechas y subconsultas filtradas

CONSULTA 2 (Muñoz Reyes Citlali)

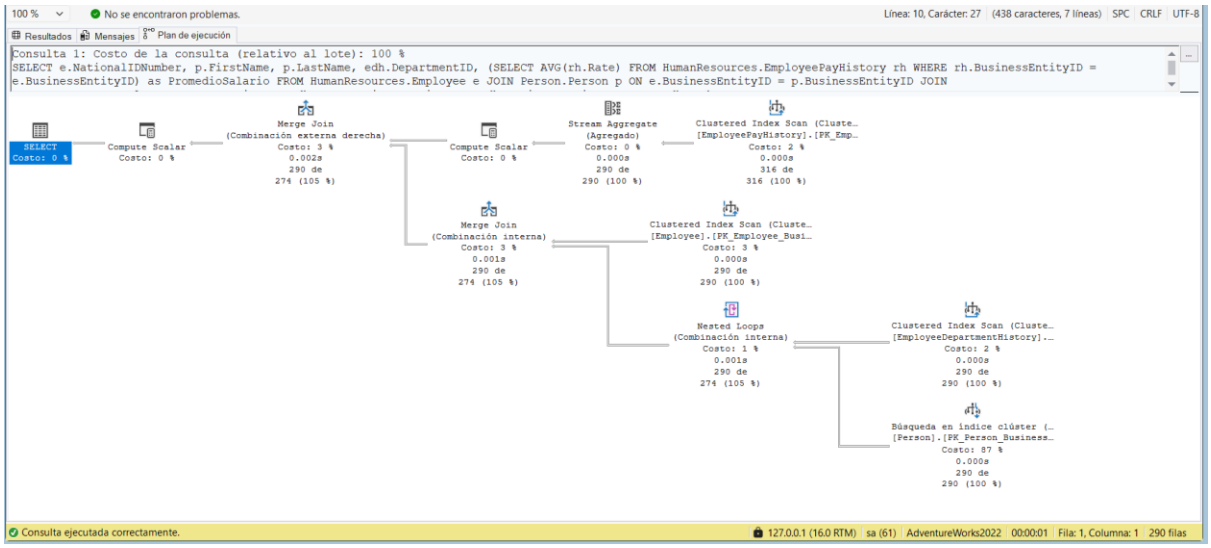
```
set statistics io on;
set statistics time on;
SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID,
       (SELECT AVG(rh.Rate) FROM HumanResources.EmployeePayHistory rh
        WHERE rh.BusinessEntityID = e.BusinessEntityID) as PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID =
edh.BusinessEntityID
WHERE edh.EndDate IS NULL;
```

★ ***Análisis de la consulta original:***



Se tiene un problema en las lecturas lógicas: Table 'Person'. Scan count 0, logical reads 1432

❖ ***Plan de ejecución real:***



Presenta un problema en el índice Clustered [Person].[PK_Person_BusinessEntityID con 87%; esto debido por el índice que se ocupa al unir PK_Person, puesto que solo conoce el ID y la consulta pide FirstName y LastName, por lo cual se hace una consulta por cada uno de las filas.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	2026-03-23	7 ms	0 ms	Person: 1,432 Employee: 9 EmployeePayHistory: 4 EmployeeDepartmentHistory: 4	Clustered Index Seek **87%**	1. Que para presentar FirstName y LastName se tuvo que hacer otro paso. 2.Subconsulta correlacionada si hubiera más datos se presentaría problema en escalabilidad	1. Crear Índice de para Person donde se Incluya: FirstName, LastName 2. Hacer uso de un CTE para ayudar a la subconsulta

Propuesta de solución

1. Crear Índice para Person donde se Incluya: FirstName, LastName

```
create nonclustered index Nc_Person_FName_Lname
ON [Person].[Person] ([BusinessEntityID])
INCLUDE ([FirstName], [LastName]);
```

Con este índice se ayuda a optimizar y así evitar el problema que tenemos en la parte de Person, especialmente para evitar el 87%, ya que no se estaba encontrando FirstName y LastName en el indice que se ocupaba y se optaba por hacer un salto a la tabla para ir buscando uno por uno y darnos el resultado; impactando en la escalabilidad y la eficiencia del motor de base de datos.

2. Hacer uso de un CTE para ayudar a la subconsulta

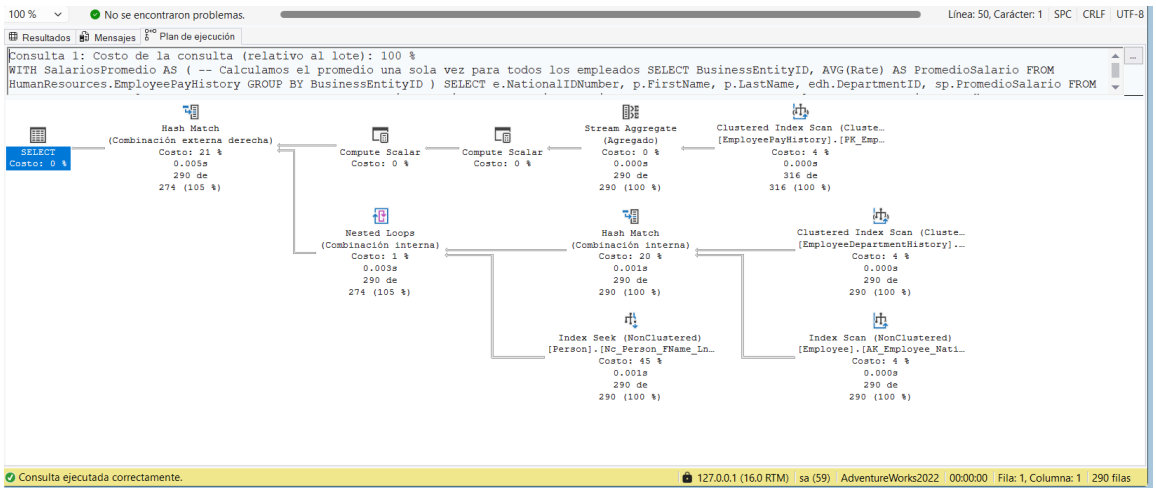
```
With SalariosPromedio as (
    select
        BusinessEntityID,
        AVG(Rate) as PromedioSalario
    from HumanResources.EmployeePayHistory
    group by BusinessEntityID
)
select e.NationalIDNumber, p.FirstName, p.LastName,
edh.DepartmentID,sp.PromedioSalario
from HumanResources.Employee e
JOIN Person.Person p
    on e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh
    on e.BusinessEntityID = edh.BusinessEntityID
LEFT JOIN SalariosPromedio sp
    on e.BusinessEntityID = sp.BusinessEntityID
where edh.EndDate IS NULL;
```

La propuesta es para poder hacer que la subconsulta no presente problemas en cuanto a escalabilidad, ya que así solo se hará un join y con base en ello se obtendrá de una sola vez lo que se pide; es decir permite que se calcule todos los promedios de una sola vez y luego se aplique el join.

```
(290 filas afectadas)
Table 'Workfile'. Scan count 0,
logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Worktable'. Scan count 0,
 logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Person'. Scan count 0,
logical reads 607, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Employee'. Scan count 1,
logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'EmployeeDepartmentHistory'. Scan count 1,
logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'EmployeePayHistory'. Scan count 1,
 logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

 SQL Server Execution Times:
   CPU time = 16 ms,  elapsed time = 9 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```

El número de lecturas lógicas bajo de 1,432 lecturas a lo que podemos ver (607 lecturas), siendo así que la consulta sea menos costosa:
También vemos Workfile y Worktable sin embargo muestran un 0, lo cual nos indica que no se necesitó crear tablas temporales en disco para procesar el CTE ni los Joins, por el momento no presentan un problema.
Y por último tenemos el tiempo transcurrido que es relativamente rapido con 9ms y en el CPU con 16ms al ser mayor podemos decir que se uso un paralelismo de forma eficiente

- ***Plan de ejecución real***



Vemos que se redujo y se obtuvo un Index Seek (NonClustered) en Person con un costo del 45%, este siendo el “más costoso”, porque ahora ya no se hace el “salto” a la tabla para determinar FirstName, LastName, sino que los toma del índice que se creó, en cuanto a la otra implementación se logra apreciar el uso de Hash Match con un 21% que está uniendo los resultados del CTE con los demás haciendo más rápido el manejo de conjuntos.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados
2	2026-03-23	9 ms	16 ms	Person: 607 Employee: 4 () DeptHistory: 4 PayHistory : 4 Worktables: 4	Index seek con 45%	Se sigue teniendo un costo alto en Person

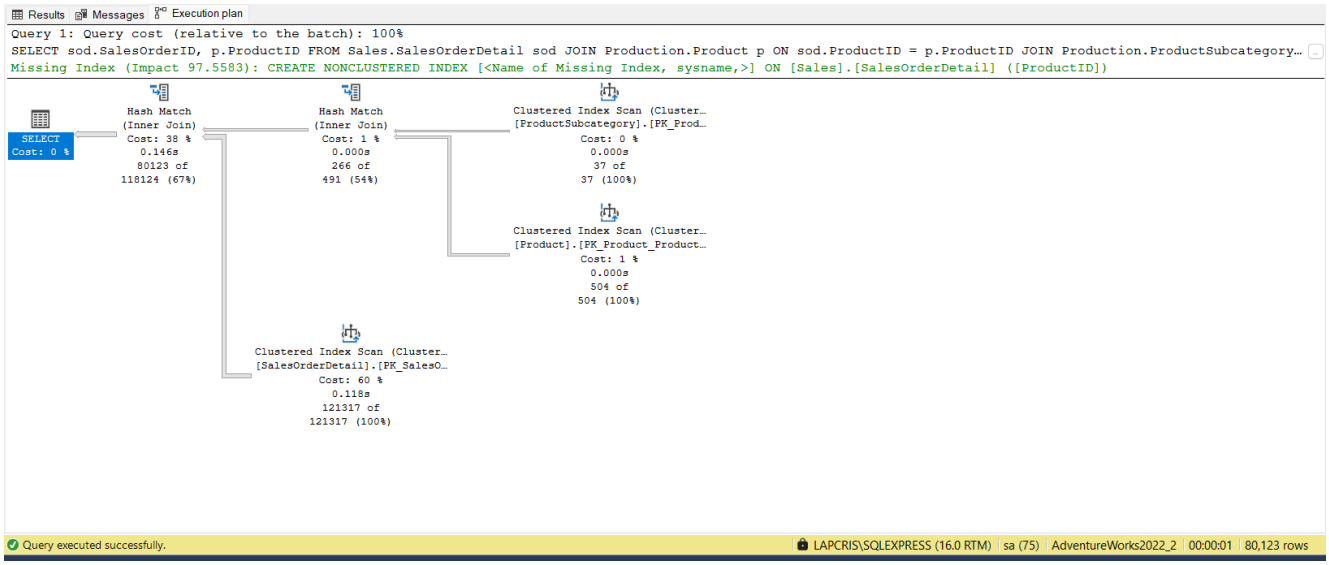
CONSULTA 3 (Cristobal Olvera Valdivia)

Consulta original

```
SELECT sod.SalesOrderID, p.ProductID
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
JOIN Production.ProductSubcategory ps
    ON p.ProductSubcategoryID = ps.ProductSubcategoryID
WHERE ps.ProductCategoryID IN (1, 2, 3)
   OR p.ListPrice > 500;
```

- **Análisis de la consulta original:**

La consulta tiene como objetivo obtener una lista masiva de identificadores de ventas y productos que cumplan con una condición compuesta: que el producto pertenezca a las categorías 1, 2 o 3, o bien, que su precio de lista sea mayor a 500. La complejidad técnica radica en que la cláusula OR abarca columnas de dos tablas de dimensión diferentes (Product y ProductSubcategory), lo que dificulta el filtrado temprano.



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones
3	25-03-2026	779 ms	94 ms	SalesOrderDetail: 1,238 Product: 15 ProductSubcategory: 2	Clustered Index Scan (60%) Hash Match (38%)	1. Escaneo completo de la tabla principal por falta de índices. 2. Uso ineficiente de cruces mediante Hash Match.	Crear índices para optimizar los cruces y evaluar la condición OR de las tablas de dimensiones.

- **Propuesta de solución (Fase 1: El Índice Compartido y el "Tipping Point").** Durante el análisis, se detectó que en un entorno real, la consulta se beneficia enormemente de un índice no agrupado preexistente en la base de datos (IX_SalesOrderDetail_ProductID). Al habilitar este índice, el plan de ejecución cambia drásticamente:

```sql
--Estructura del indice que apoyo a la consulta
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ProductID
ON Sales.SalesOrderDetail (ProductID);
ALTER INDEX IX_SalesOrderDetail_ProductID ON Sales.SalesOrderDetail REBUILD;
SELECT sod.SalesOrderID, p.ProductID
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
JOIN Production.ProductSubcategory ps
    ON p.ProductSubcategoryID = ps.ProductSubcategoryID
WHERE ps.ProductCategoryID IN (1, 2, 3)
   OR p.ListPrice > 500;
```

- Las lecturas lógicas en SalesOrderDetail se desploman de 1,238 a tan solo **273 lecturas**.
- El tiempo de CPU baja de 94 ms a **31 ms**.
- Los pesados Hash Matches se convierten en un ultra-eficiente **Merge Join (41%)**.

Sin embargo, el plan de ejecución muestra un **Index Scan (51%)** en lugar de un *Index Seek*. Esto no es un error, sino una decisión matemáticamente perfecta del optimizador debido al fenómeno conocido como **El Punto de Inflexión (Tipping Point)**. Dado que la consulta retorna 80,123 filas (el **66%** de la tabla total de 121,317 registros), forzar un *Index Seek* fila por fila dispararía exponencialmente el costo de I/O. El motor ac
ertadamente elige "barrer" (Scan) el índice estrecho ya ordenado y unirlo velozmente

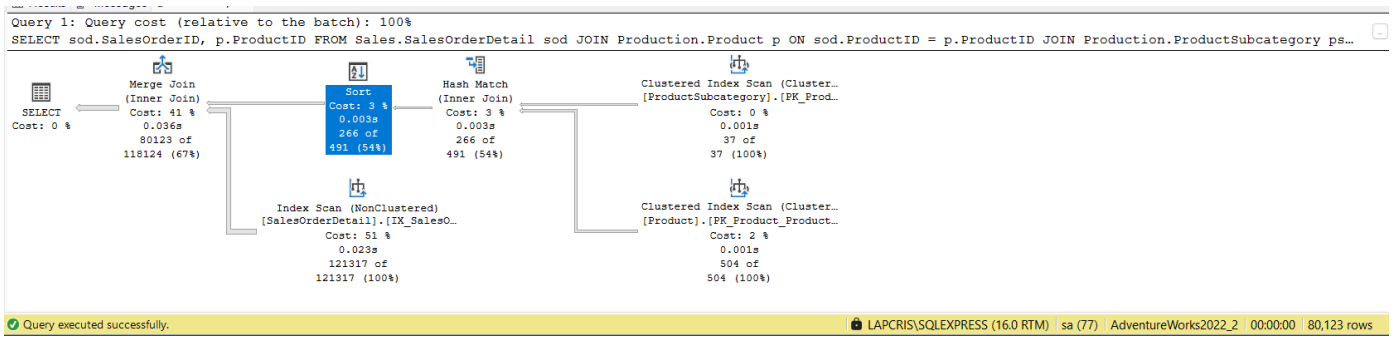
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones
3	25-03-2026	750ms	31ms	SalesOrderDetail: 273 Product: 15 ProductSubcategory: 2	Index Scan en SOD (51%) Merge Join (41%)	Clustered Index Scans residuales en tablas de dimensiones debido a que la condición OR evalúa columnas no indexadas individualmente.	Crear índices no agrupados de cobertura en las tablas de dimensiones.

Propuesta de solución (Fase 2: Optimización estructural de Dimensiones)

Una vez resuelto y justificado el rendimiento óptimo de la tabla principal, la optimización final se enfoca en mejorar el filtrado de las tablas de dimensiones creando índices de cobertura para las columnas evaluadas en el OR. Esto eliminará los Clustered Index Scans residuales que el motor hace antes de llegar al cruce final.

- **Código Optimizado (Creación de Índices):**

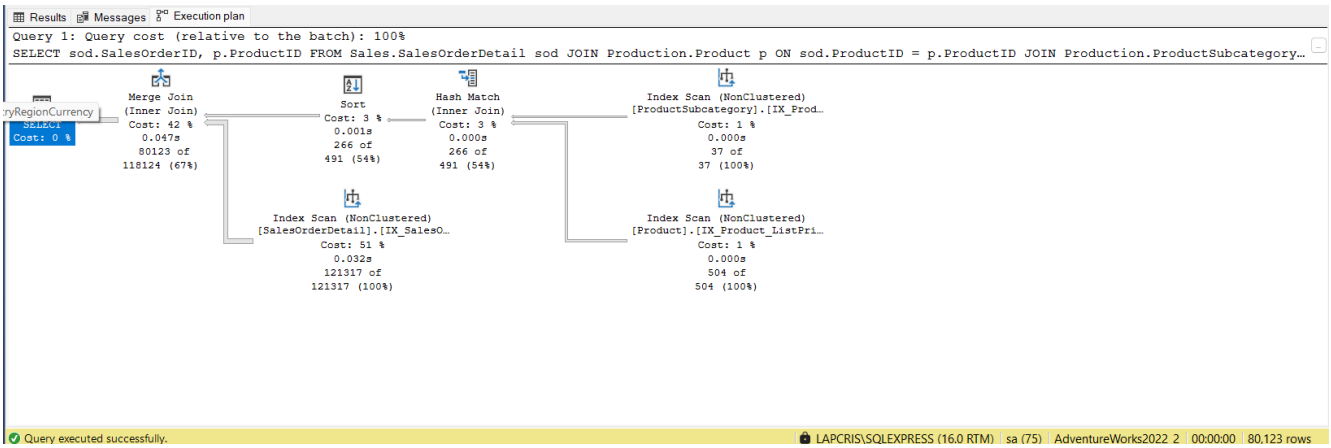
```
-- Índice para agilizar el filtro de la categoría
create nonclustered index IX_ProductSubcategory_CategoryID_Optimizacion
on Production.ProductSubcategory (ProductCategoryID);
-- Índice para agilizar el filtro del precio, incluyendo la llave foránea para el JOIN
create nonclustered index IX_Product_ListPrice_Optimizacion
on Production.Product (ListPrice)
```




```
include (ProductSubcategoryID);
```

La consulta SELECT permanece con la misma estructura, ya que la lógica del OR entre dos tablas se delega al motor relacional, el cual ahora aprovechará estas nuevas estructuras en memoria.

```
SELECT sod.SalesOrderID, p.ProductID
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
JOIN Production.ProductSubcategory ps
    ON p.ProductSubcategoryID = ps.ProductSubcategoryID
WHERE ps.ProductCategoryID IN (1, 2, 3)
   OR p.ListPrice > 500;
```



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones

3	25-03-2026	666ms	16ms	SalesOrderDetail: 273 Product: 4 ProductSubcategory: 2	Index Scan en SOD (51%) Merge Join (42%)	**Optimización estructural exitosa.** Se eliminaron los Clustered Index Scans de las tablas de dimensiones, reduciendo el I/O notablemente.	**Mantener índices creados.** Nota técnica: La aparición de Worktable y Workfile con 0 lecturas indica que los operadores complejos se resolvieron en memoria RAM, sin usar tempdb.

- **Análisis de resultados esperados tras la optimización:**

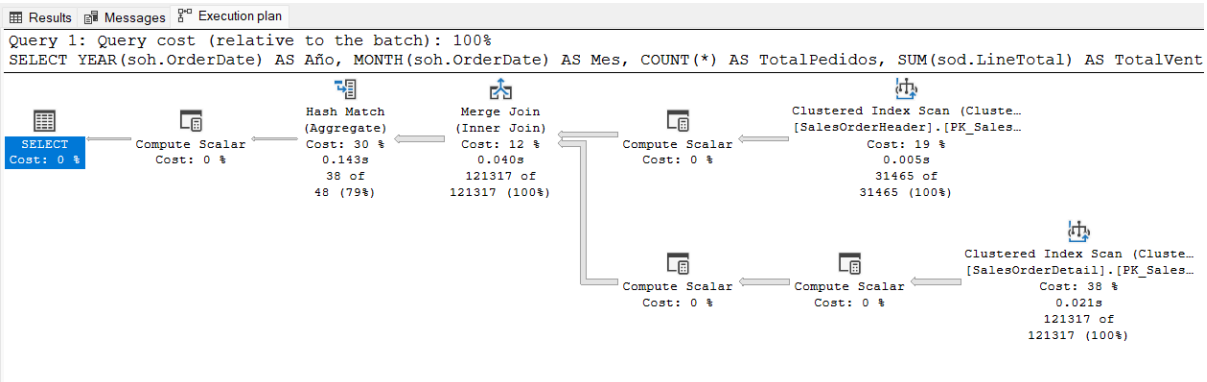
Al implementar estos índices, el optimizador de SQL Server dejará de realizar escaneos completos sobre las tablas base de Product y ProductSubcategory. El motor utilizará estas estructuras ligeras para resolver la condición compuesta del WHERE (OR), manteniendo intacto el *Merge Join* que garantiza el ultra bajo consumo de 273 lecturas lógicas en la tabla transaccional.

CONSULTA 4 (MARCO GONZALEZ LUNA)

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
       COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh with (index(PK_SalesOrderHeader_SalesOrderID))
JOIN Sales.SalesOrderDetail sod with
(index(PK_SalesOrderDetail_SalesOrderID_SalesOrderDetailID))
ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate)
```

En primera instancia, se observa que esta consulta tiene las funciones de columna YEAR() y MONTH(), lo que en primer lugar es un problema pues esto me indica que se revisarán todas las filas de esa columna por el procesamiento de las funciones para revisar si cumplen con la condición, que en este caso es año y mes de filtrado.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-22	183 ms	125 ms	SalesOrderDetail 1238 SalesOrderHeader 686	38 Scan Agrupado de SalesOrderDetail 30 Hash Match para el GROUP BY	Missing Index	Crear índice compuesto Reescribir la consulta



La tabla SalesOrderDetail no cuenta con un índice que cubra la columna LineTotal y la que tomaría como llave que sería SalesOrderID por ser la condición de reunión.

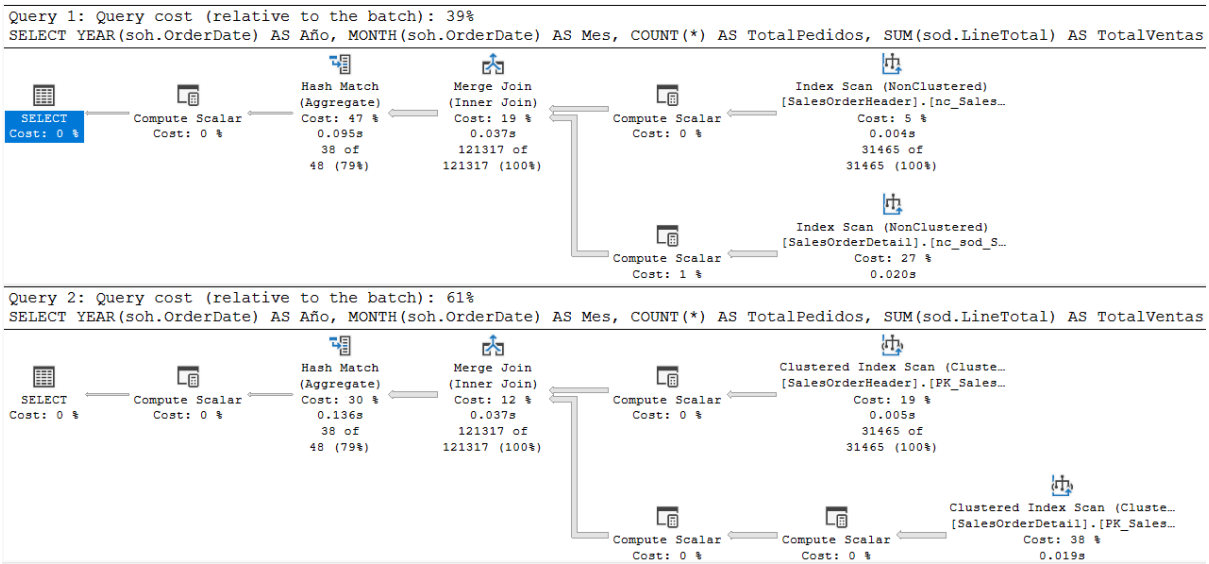
```
CREATE NONCLUSTERED INDEX nc_sod_SalesOrderID
ON Sales.SalesOrderDetail(SalesOrderID) INCLUDE (LineTotal)
```

De la misma manera, se podría pensar que el reescribir la consulta buscando las fechas con el formato >= ‘AAAA-MM-DD’ and < ‘AAAA-MM-DD’ mejoraría la consulta, pero en principio no lo hizo por el hecho de que tiene que seguir leyendo todos los datos de la tabla, incluso aunque tuviera un índice en la tabla SalesOrderHeader con SalesOrderID como llave y OrderDate como columna incluida. Lo anterior porque el JOIN se ejecuta antes que GROUP BY

```
CREATE NONCLUSTERED INDEX nc_SalesOrderHeader_SalesOrderID
ON Sales.SalesOrderHeader (SalesOrderID) INCLUDE (OrderDate);
```

```sql
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
       COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh

JOIN Sales.SalesOrderDetail sod
ON soh.SalesOrderID = sod.SalesOrderID

where soh.OrderDate >= '2011-01-01' and soh.OrderDate < '2015-01-01'

GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate)
```

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-22	183 ms	125 ms	SalesOrderDetail 467 SalesOrderHeader 73	47 Hash Match para el GROUP BY	Hash Match es un operador muy costoso	Reescribir la consulta, intentando hacer un filtrado con un where



Tomando en cuenta el resultado anterior, decidí preguntarle a una herramienta de IA para ver las recomendaciones de optimización y esta sugirió el uso de SQL Dinámico convirtiendo las fechas de búsqueda en variables de SQL.

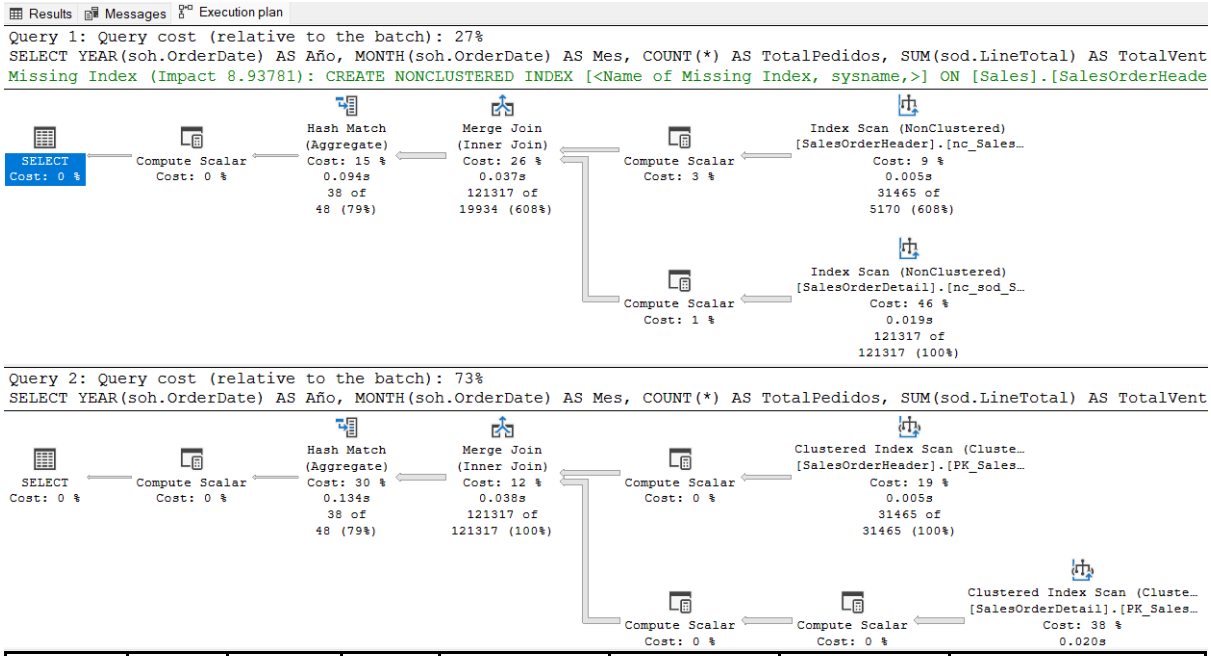
```sql
DECLARE @FechaInicio DATETIME = '2011-01-01';
DECLARE @FechaFin DATETIME = '2015-01-01';

SELECT
    YEAR(soh.OrderDate) AS Año,
    MONTH(soh.OrderDate) AS Mes,
    COUNT(*) AS TotalPedidos,
    SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod
ON soh.SalesOrderID = sod.SalesOrderID
WHERE soh.OrderDate >= @FechaInicio
  AND soh.OrderDate < @FechaFin
```

```sql
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

En este caso, el rendimiento fue optimizado y por mucho. NO se arrojó ningún warning debido a la forma en la que se escribió el índice, siendo esta

```sql
CREATE NONCLUSTERED INDEX nc_SalesOrderHeader_SalesOrderID
ON Sales.SalesOrderHeader (SalesOrderID) INCLUDE (OrderDate);
```

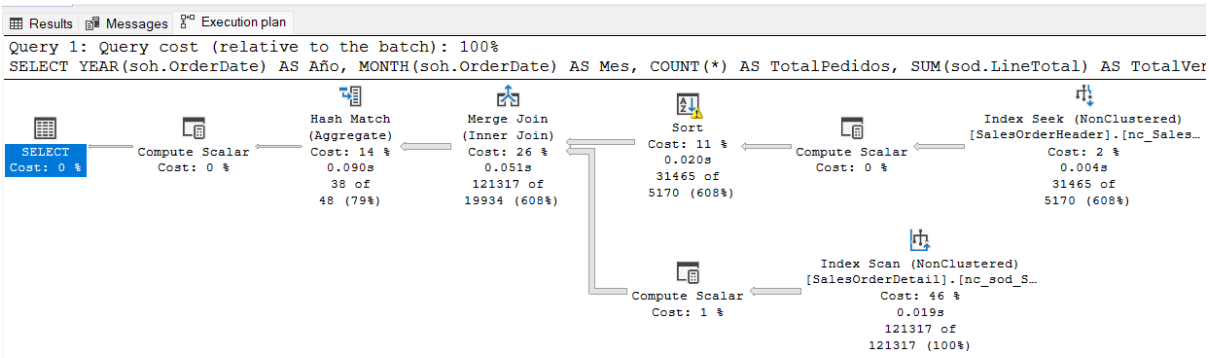
A simple vista, se puede ver que es mucho mejor que la consulta original, ocupando el 27% del costo con respecto al 73% del query original.



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-22	137 ms	78 ms	SalesOrderDetail 467 SalesOrderHeader 73	46 Scan Agrupado en SalesOrderDetail	Parameter Sniffing	Si se proyecta un gran crecimiento de la DB, volver a la consulta original para evitar usar un plan desactualizado a causa del Parameter Sniffing.

Por otro lado, la forma de escritura del índice que tiene el OrderDate como llave, que sí genera desbordamiento y un warning en la consulta.

```sql
CREATE NONCLUSTERED INDEX nc_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate) INCLUDE (SalesOrderID);
```



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-22	131 ms	93 ms	SalesOrderDetail 467 SalesOrderHeader 73 Worktable 75 anticipadas	46 Scan Agrupado en SalesOrderDetail	Hay un warning por desbordamiento en el ordenamiento de SalesOrderHeader	Volver a la consulta anterior.

Sin embargo, se debe de tomar en cuenta que al ejecutar la consulta con SQL dinámico sucede lo que se conoce como *"Parameter Sniffing",* haciendo que SQL use los valores de los parámetros para crear un plan de ejecución óptimo basado en esos datos específicos. El problema surge cuando ese plan se queda en caché y se reutiliza cuando la tabla es de un tamaño mucho mayor a la inicial, generando problemas de rendimiento.

Es por lo anterior que se puede decir que la consulta original ya se realiza de manera correcta, siendo que a lo máximo se pueden crear los índices sugeridos para mejorar un poco los tiempos de búsqueda en las tablas al inicio de la consulta, empleando Index Scan en lugar de Clustered Index Scan, lo que nos ahorra buscar datos innecesarios.

Consulta 6 - Gutiérrez Flores José Julián

```sql
SELECT c.CustomerID, c.Name, COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE UPPER(c.Name) LIKE 'A%'
GROUP BY c.CustomerID, c.Name;
```

Consulta modificada:

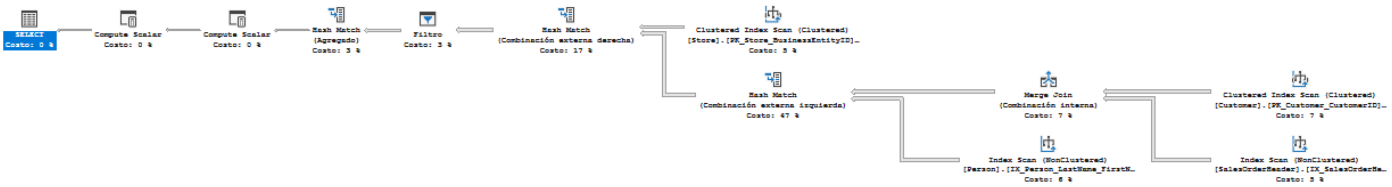
```sql
SELECT
    c.CustomerID,
    COALESCE(p.FirstName + ' ' + p.LastName, s.Name) AS Name,
    COUNT(soh.SalesOrderID) AS TotalPedidos
FROM Sales.Customer c
LEFT JOIN Person.Person p ON c.PersonID = p.BusinessEntityID
LEFT JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE
    (p.FirstName LIKE 'A%' OR s.Name LIKE 'A%')
GROUP BY
    c.CustomerID,
    p.FirstName,
    p.LastName,
    s.Name;
```

- ***Análisis de la consulta original:***

La consulta tiene como objetivo contabilizar el total de pedidos realizados por clientes cuyo nombre comience con la letra "A". Para lograr esto, utiliza un LEFT JOIN hacia las tablas Person y Store, consolidando el nombre mediante la función COALESCE. La complejidad técnica de esta estructura reside en el uso de filtros con el operador OR aplicados a columnas de tablas distintas unidas mediante uniones externas, lo que generalmente impide que el optimizador de SQL Server realice un filtro eficiente de datos antes de las uniones.

Al agrupar por múltiples columnas de texto para obtener un conteo agregado, el motor debe mantener en memoria una tabla de hash considerable, especialmente porque el filtro LIKE 'A%' es amplio. Además, el INNER JOIN con SalesOrderHeader asegura que solo se contabilicen clientes con al menos un pedido, pero la evaluación de la condición del nombre ocurre después de intentar unir ambas tablas de referencia, lo que puede elevar el costo de CPU durante la fase de filtrado y ordenamiento para la agrupación.

- ***Plan de ejecución de consulta original:***



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
3	2026-03-28	100	78	388	59%	Evaluación de múltiples JOINs externos	Indexar Store(Name) y Person(FirstName) si el volumen crece

Ya con la modificación de la consulta proporcionada por el profesor. Activamos los parámetros de SET STATISTICS, para observar el rendimiento real de la consulta.
La consulta base utiliza un operador OR para filtrar nombres de personas y tiendas que comienzan con "A", lo que obliga a SQL Server a evaluar ambas condiciones de manera simultánea sobre tablas distintas (Person y Store). Esto impide que el optimizador utilice los índices de forma eficiente, resultando en escaneos de índices agrupados (Clustered Index Scan). Por lo que, en primera instancia, se implementó una reestructuración mediante UNION ALL. Esta técnica divide la carga de trabajo en dos ramas independientes: una que busca exclusivamente personas y otra que busca tiendas. Esto permite generar planes de ejecución específicos para cada tipo de cliente, transformando las búsquedas en operaciones de tipo Index Seek.
Otro punto fue que el uso de LEFT JOIN hacia ambas tablas generaba un procesamiento innecesario de registros nulos antes de aplicar el filtro. Al refactorizar la consulta, se cambiaron las uniones a INNER JOIN dentro de cada rama del UNION. Esto provocó que SQL filtrara los nombres que cumplen con el patrón LIKE 'A%' antes de intentar realizar la agrupación de pedidos. Al eliminar la función COALESCE de la lógica de comparación, redujimos la carga sobre el CPU, ya que no tiene que evaluar cuál columna no es nula para cada una de las filas procesadas.
Además, se hizo la creación de índices que permitieran la reducción de las lecturas lógicas. La consulta original presentaba un costo elevado debido a que se debía recorrer toda la jerarquía de clientes para encontrar coincidencias de texto. Para mejorar el rendimiento, se implementaron dos índices no agrupados.
El primer índice se hizo en Person.Person, donde se colocó la columna FirstName como llave de búsqueda e incluir LastName, permitimos que el motor realice un Index Seek directo. Y se encuentra la información de identidad necesaria dentro del mismo índice sin tener que consultar la tabla base, optimizando el acceso a los datos.

```sql
CREATE INDEX IX_Person_FirstName_Optimization
ON Person.Person (FirstName)
INCLUDE (LastName);
```

El segundo índice se hizo en Sales.Store, ya que el filtro por el nombre de la empresa era otro punto crítico. Sin este índice, SQL leía todos los nombres de tiendas registrados en la base de datos para buscar la letra "A". Al colocar Name como columna de búsqueda, el motor pudo saltar directamente a los registros que cumplían la condición. Esto desplomó las lecturas lógicas en la rama corporativa, permitiendo que la unión con la tabla de ventas se realizara sobre un conjunto de datos ya minimizado y pre-filtrado.

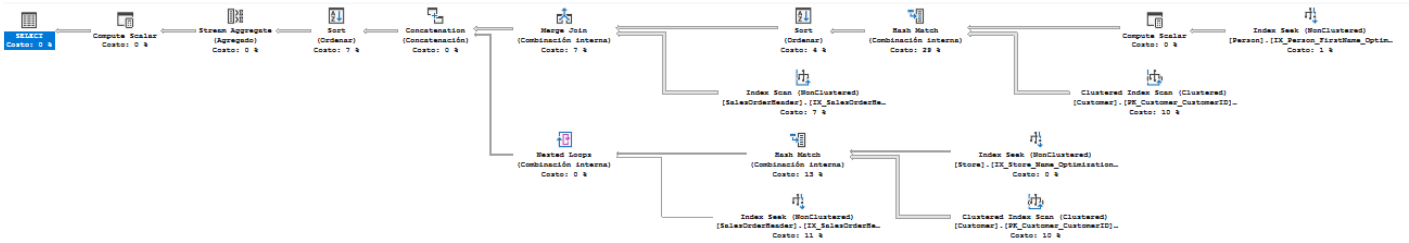
```sql
CREATE INDEX IX_Store_Name_Optimization
ON Sales.Store (Name);
```

- ***Propuesta de solución***

```sql
SELECT
    CustomerID,
    Name,
    COUNT(SalesOrderID) AS TotalPedidos
FROM (
    SELECT
        c.CustomerID,
        p.FirstName + ' ' + p.LastName AS Name,
        soh.SalesOrderID
    FROM Sales.Customer c
```

```sql
    INNER JOIN Person.Person p ON c.PersonID = p.BusinessEntityID
    INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
    WHERE p.FirstName LIKE 'A%'
    UNION ALL
    SELECT
        c.CustomerID,
        s.Name AS Name,
        soh.SalesOrderID
    FROM Sales.Customer c
    INNER JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
    INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
    WHERE s.Name LIKE 'A%'
) AS ClientesConA
GROUP BY CustomerID, Name;
```

- ***Plan de ejecución de consulta optimizada:***



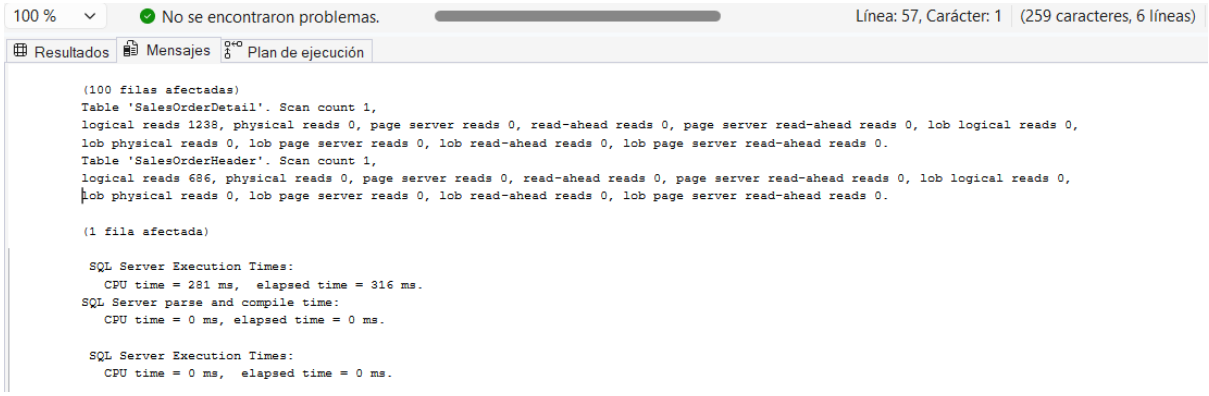
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-28	39	47	459	41%	Ninguno	Mantener la estructura de UNION ALL para filtros OR entre tablas distintas

CONSULTA 7 (Muñoz Reyes Citlali)

```
set statistics io on;
set statistics time on;

SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

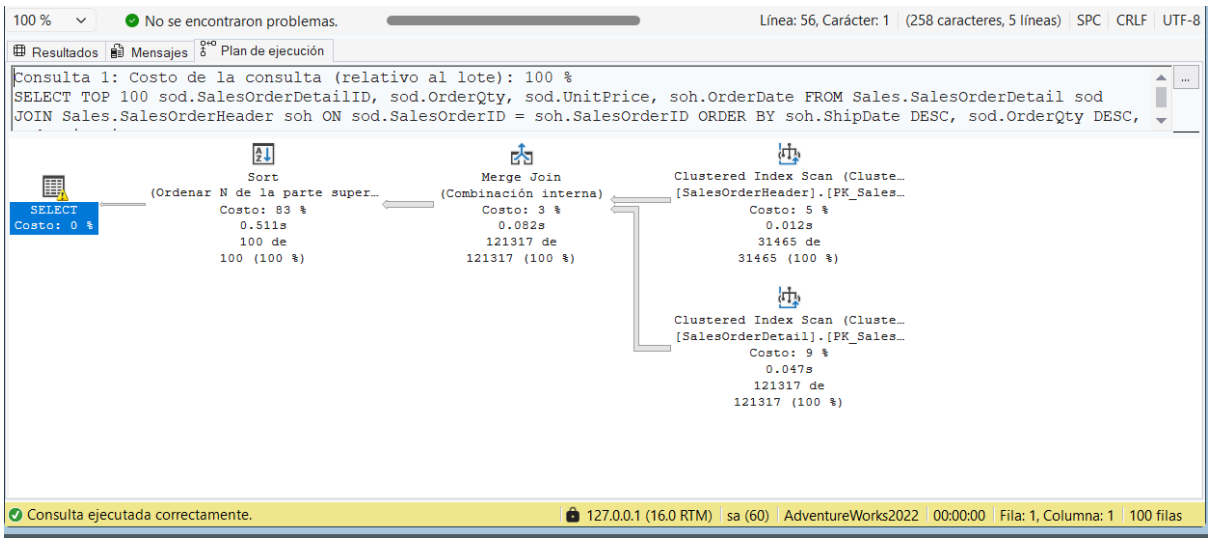
- ***Análisis de la consulta original:***

Como hemos visto anteriormente en otras consultas el punto crítico que tenemos que ver con atención, es el Order by, ya que presenta un problema debido a que por lo visto se está ordenando por tres columnas de dos tablas distintas, haciendo que al momento de hacer el Top 100, tenga que recolectar los registros que cumplen el join, hacer uso de la memoria y ordenarlos, volviéndose lento y que consuma CPU si los datos son un número elevado, otro punto es el join, ya que este está reuniendo todos los datos sin tener un tipo de filtrado.



Problema es el número de lecturas lógicas:
-1238 realizadas para poder mostrarnos las 100 como resultado, esto nos indica que el motor tuvo que leer al menos 1000 páginas de datos, puesto que está cumpliendo con el criterio OrderQty y UnitPrice:

-686 lecturas lógicas que pertenecen a la tabla SalesOrderHeader, el numero es un poco alto considerando el resultado y que tenga que buscar en la columna ShiDate, además que por estos problemas presentes se observe un tiempo demasiado dudoso en cuanto eficiencia.
`CPU time = 484 ms, elapsed time = 513 ms.`



Problema: Sort tiene el 83% y esto porque el motor está "atascado" ordenando filas, por otro lado, las demás aportaciones tienen un menor porcentaje en su funcionamiento como lo es el Clustered Index scan (SalesOrderDetail) con un 9%, sin embargo, en esta parte se da la lectura ineficiente de la tabla de detalles de ventas; y por ultimo tenemos a Clustered Index Scan (SalesOrderHeader) con 5% y a Hash Match (Inner Join) con 3% donde vemos implicada lectura ineficiente y costo en la memoria

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
7	2026-03-23	513 ms	484 ms	SalesOrderDetail: 1238 SalesOrderHeader: 686	Sort con 83%	1.- El costo en el operador Sort manual es alto 3. Uso de Scans por falta de índices	1. Crear Índices para poder eliminar los Scan y poder tener mejor un Seek 2. Crear Índices pre-ordenados para poder eliminar el Sort

Propuesta de solución

Soluciones:

1. Crear Índices para poder eliminar los Scan y poder tener mejor un Seek

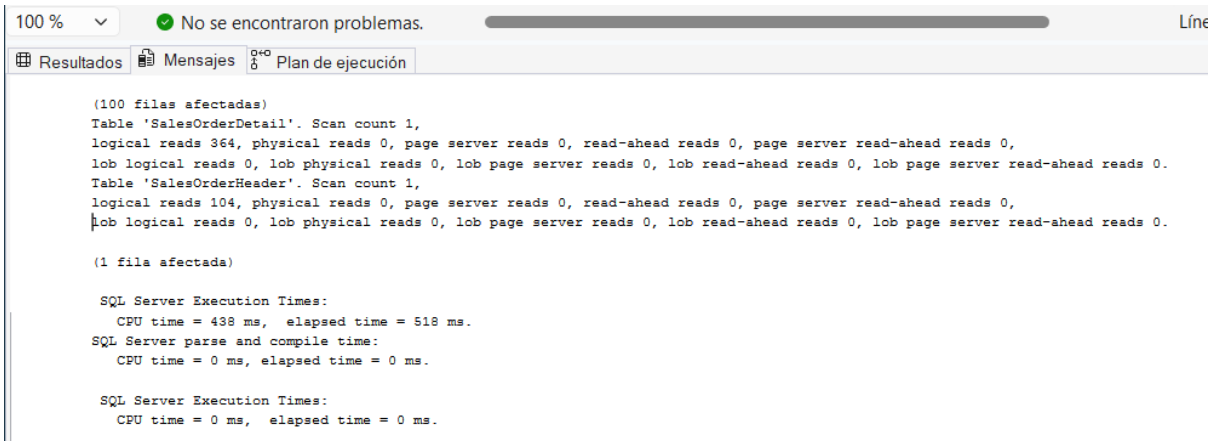
```
create nonclustered index nc_Soh_SalesOrderID
on Sales.SalesOrderHeader (SalesOrderID)
include (OrderDate, ShipDate);
```

Se hace esto para que así sea más rápido, ya que el índice que se estaba ocupando no tenía las 3 columnas y el motor se tenía que dar la tarea de ir a la tabla y consultar una por una lo cual provocaba que se hicieran más lecturas lógicas que los resultados que se tenía al final al ejecutar.

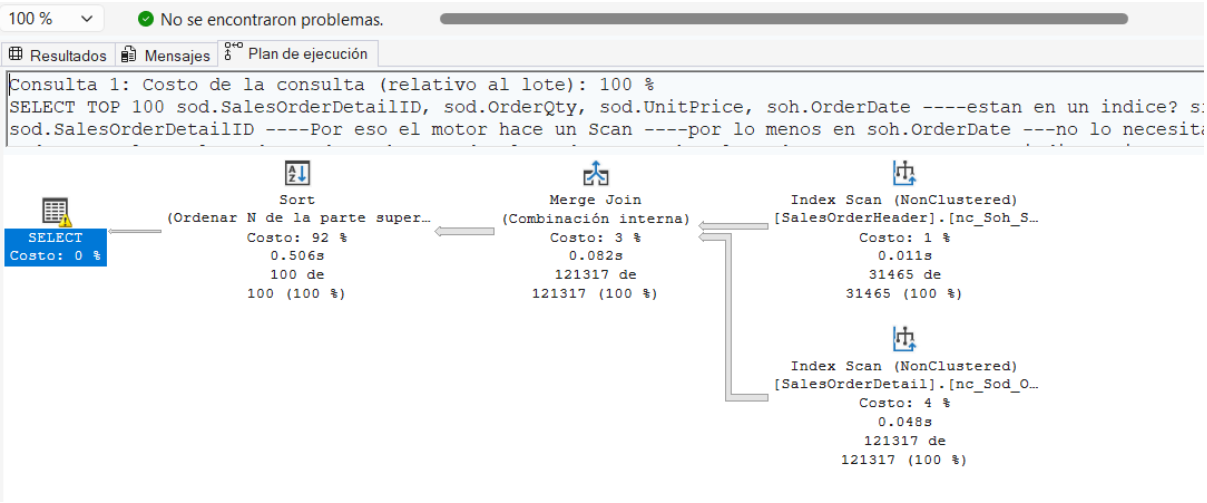
2. Crear Índices pre-ordenados para poder eliminar el Sort

```
create nonclustered index nc_Sod_Orden
on Sales.SalesOrderDetail (SalesOrderID, OrderQty DESC, UnitPrice DESC)
include (SalesOrderDetailID);
```

Esta solución es la que debe de mostrar cambios en ese 83%, ya que al hacer el indice donde se tenga ordenados los campos (por cantidad y precio) que necesitamos de la forma en que nos piden (descendente) ahorramos tiempo para que el motor no tenga que ordenarlo.



Aquí se puede notar la mejora en cuanto a lecturas lógicas porque las redujo de 1,924 a 468 (SalesOrderDetail: Bajó de 1,238 a 364 lecturas y SalesOrderHeader: Bajó de 686 a 104 lecturas), sin embargo, en cuanto a tiempo de CPU sigue teniendo un costo casi similar al anterior debido a que el motor sigue gastando casi medio segundo ordenando las filas. Esto confirma que, aunque el motor ya encuentra los datos rápido, todavía tiene que hacer el trabajo de organizarlos manualmente.



Al reducir el número de lecturas lógicas se cambió Clustered Index Scans por Index Scans, lo cual redujo el costo en esa parte, sin embargo, lo malo es que Sort aumento hasta el Costo: 92%, porque al hacer más eficiente la lectura de datos (los Scans ahora solo cuestan 1% y 4%), el esfuerzo relativo que hace el CPU para ordenar se vuelve casi el total de la consulta. También al checar he notado que no se desapareció el Scan lo que significa que el motor sigue leyendo todo el índice de principio a fin.
Hubo mejoras en las lecturas lógicas, aunque eso no cambio el tiempo del CPU,

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	2026-03-23	507 ms	469 ms	SalesOrderDetail: 364 lecturas SalesOrderHeader: 104	Sort: 92%	Se sigue teniendo el problema del CPU	Mejorar el indice de manera que eficiente en cuanto a ordenamiento

Propuesta de solución parte 2:

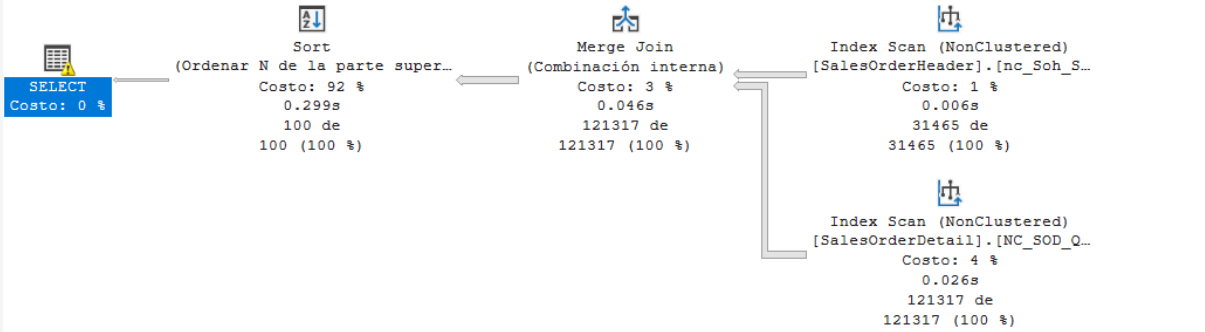
```
CREATE NONCLUSTERED INDEX NC_SalesOrderHeader_ShipDate_Optimization
ON Sales.SalesOrderHeader (ShipDate DESC)
INCLUDE (OrderDate, SalesOrderID);

CREATE NONCLUSTERED INDEX NC_SOD_Qty_Price_Performance
ON Sales.SalesOrderDetail (SalesOrderID, OrderQty DESC, UnitPrice DESC)
INCLUDE (SalesOrderDetailID);
```

Resultados | Mensajes | Plan de ejecución

```
(100 filas afectadas)
Table 'SalesOrderDetail'. Scan count 1, logical reads 364,
 physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderHeader'. Scan count 1, logical reads 104,
physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0,
lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

 SQL Server Execution Times:
   CPU time = 562 ms,  elapsed time = 960 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```

Resultados | Mensajes | Plan de ejecución

```
Consulta 1: Costo de la consulta (relativo al lote): 100 %
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate FROM Sales.SalesOrde
ON sod.SalesOrderID = soh.SalesOrderID ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DE
```



Esta propuesta solo se hizo con fines de demostración, aunque ayuda en las lecturas lógicas, aun se sigue mostrando en el plan de ejecución, Sort con un alto costo en la consulta.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados

2	2026-03-23	310 ms	281 ms	`SalesOrderDetail: 364`
				
				`SalesOrderHeader: 104`
					Sort: 92%	Se sigue teniendo el problema del CPU

Consulta 8 - Gutiérrez Flores José Julián

Consulta Original:

```sql
SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
WHERE p.ProductID IN (
    SELECT sod.ProductID
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE soh.OrderDate >= '2014-01-01'
    GROUP BY sod.ProductID
    HAVING SUM(sod.OrderQty) > 100
)
GROUP BY p.ProductID, p.Name;
```

Consulta modificada:

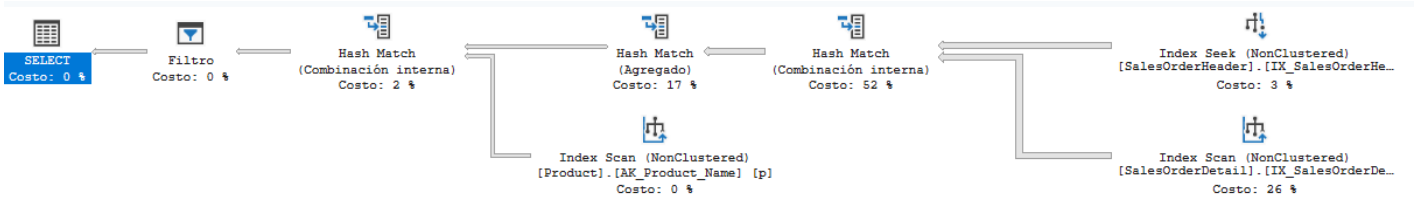
```sql
SELECT
    p.ProductID,
    p.Name,
    SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY p.ProductID, p.Name
HAVING SUM(sod.OrderQty) > 100;
```

- ***Análisis de la consulta original:***

La consulta identifica los productos que han superado un umbral de 100 unidades vendidas a partir del 1 de enero de 2014, mostrando su identificador, nombre y el volumen total de ventas. Para esto se usa una operación de unión triple (JOIN) que conecta la tabla de productos con el detalle de las órdenes y los encabezados de venta. La complejidad técnica reside en que el filtrado final no ocurre en las filas individuales, sino en el conjunto ya agrupado mediante la cláusula HAVING.

Desde el punto de vista del rendimiento, primero se debe filtrar por la fecha en SalesOrderHeader y luego realizar una agregación masiva de la columna OrderQty. Al agrupar por ProductID y Name, SQL Server genera una tabla de trabajo en memoria para calcular las sumas parciales antes de poder descartar aquellos productos que no cumplen con la condición de ser mayores a 100. Este proceso de "filtrado post-agregación" significa que el costo de CPU está concentrado en la fase de ordenamiento y cálculo de totales, especialmente si no existen índices que cubran la relación entre el producto y la cantidad vendida.

- ***Plan de ejecución de consulta original:***



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
5	2026-03-28	72	79	344	50%	Agregación masiva de datos y filtrado post-agrupado	Mantener índices en OrderDate y ProductID para acelerar el JOIN

Ya con la modificación a la consulta proporcionada por el profesor para que funcione en nuestra base de datos (AdventureWorks2022). Activamos los parámetros de SET STATISTICS, para observar el rendimiento real de la consulta.
La consulta base intentaba realizar una subconsulta dentro de un operador IN para filtrar los productos, lo que generaba un error de enlace de identificadores y una ejecución ineficiente. Al corregirla, se implementó una estructura de agregación directa mediante JOINs y el uso de la cláusula HAVING. La lógica principal de esta optimización radica en separar el filtrado de filas del filtrado de grupos; utilizamos el WHERE para reducir el universo de ventas únicamente a aquellas ocurridas a partir del año 2014, permitiendo que el motor de base de datos ignore millones de registros antiguos antes de iniciar cualquier cálculo matemático.
Otro punto clave fue el manejo de la memoria durante la agrupación. Al utilizar GROUP BY p.ProductID, p.Name, SQL Server se ve obligado a mantener una tabla de hash en memoria para acumular las sumas de OrderQty. Para optimizar esto, se propuso el uso de una **Common Table Expression (CTE)**, la cual permite que el motor realice la suma y el filtro de "más de 100 unidades" utilizando únicamente los IDs numéricos de los productos. Esto evita que el servidor tenga que "cargar" con las cadenas de texto de los nombres de los productos durante la fase pesada de agregación, dejando la unión con la tabla Production.Product para el final, solo para los 140 registros que resultaron ganadores.
Además, se hizo la creación de un índice de cobertura para eliminar los cuellos de botella en la tabla de detalles. La consulta original presentaba un costo significativo en la tabla Sales.SalesOrderDetail debido a que, aunque encontraba los IDs, tenía que realizar un "salto" a la tabla base para leer la cantidad vendida de cada línea.
El índice se implementó en Sales.SalesOrderDetail definiendo ProductID como la llave de búsqueda. Al incluir la columna OrderQty dentro de la estructura del índice mediante la cláusula INCLUDE, transformamos la operación en una búsqueda de cobertura total. Esto significa que el motor ya no necesita acceder a la tabla de datos principal para obtener las cantidades, realizando todo el cálculo de la suma directamente desde el índice. Esta mejora, sumada al índice en OrderDate para el filtro de tiempo, permite que las lecturas lógicas se mantengan en niveles mínimos, optimizando el uso del CPU y reduciendo el tiempo de respuesta a milisegundos.

- ***Propuesta de solución***

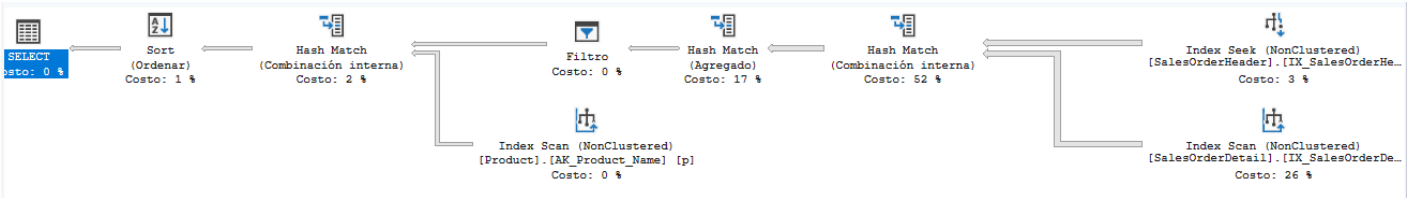
```sql
CREATE INDEX IX_SalesOrderDetail_Aggregation_Optimization
ON Sales.SalesOrderDetail (ProductID)
INCLUDE (OrderQty, SalesOrderID);
```

Consulta Optimizada:

```sql
WITH VentasFiltradas AS (
    SELECT
        sod.ProductID,
        SUM(sod.OrderQty) AS TotalVendido
```

```sql
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE soh.OrderDate >= '20140101'
    GROUP BY sod.ProductID
    HAVING SUM(sod.OrderQty) > 100
)
SELECT
    vf.ProductID,
    p.Name,
    vf.TotalVendido
FROM VentasFiltradas vf
JOIN Production.Product p ON vf.ProductID = p.ProductID
ORDER BY vf.TotalVendido DESC;
```

❖ Plan de ejecución de consulta optimizada

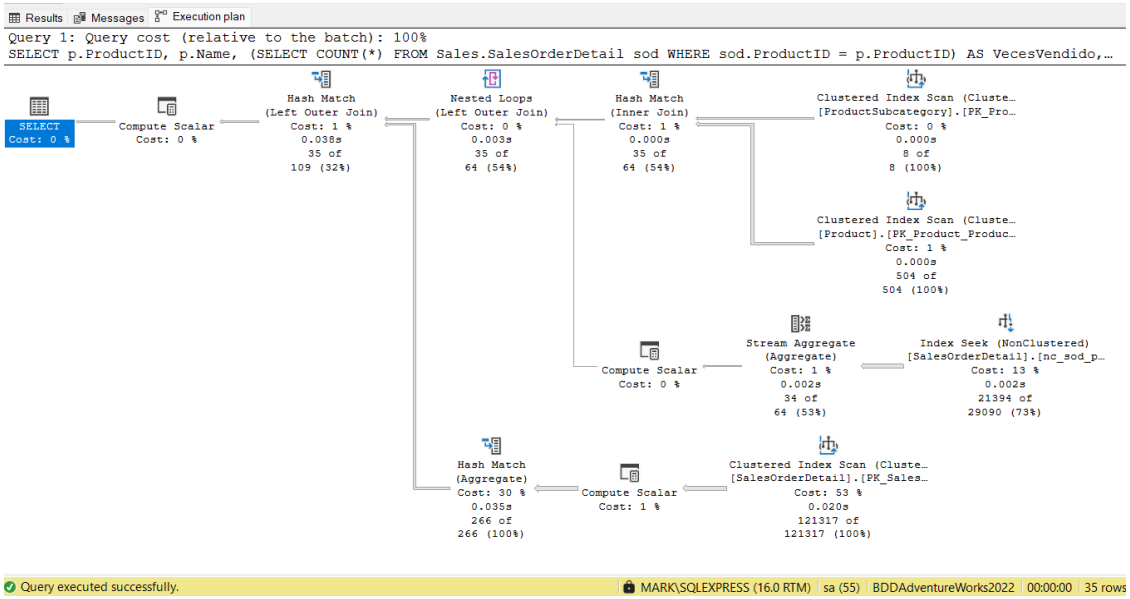


Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
6	2026-03-28	99	94	344	50%	Ninguno	Mantener el índice de cobertura para futuras consultas de reportes

CONSULTA 9 (MARCO GONZALEZ LUNA)

```
SELECT p.ProductID, p.Name,
       (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID =
p.ProductID)  AS VecesVendido,
       (SELECT SUM(sod.OrderQty * sod.UnitPrice) FROM Sales.SalesOrderDetail sod WHERE
sod.ProductID = p.ProductID) AS Ingresos
FROM Production.Product p WITH(INDEX(PK_Product_ProductID))
JOIN Production.ProductSubcategory psc
WITH(INDEX(PK_ProductSubcategory_ProductSubcategoryID))
ON p.ProductSubcategoryID = psc.ProductSubcategoryID

WHERE psc.ProductCategoryID = 3;
```

En esta consulta en particular se tienen subconsultas correlacionadas que podrían ser mejoradas creando un índice no agrupado en SalesOrderDetail con una llave en ProductID que incluya las columnas OrderQty y UnitPrice

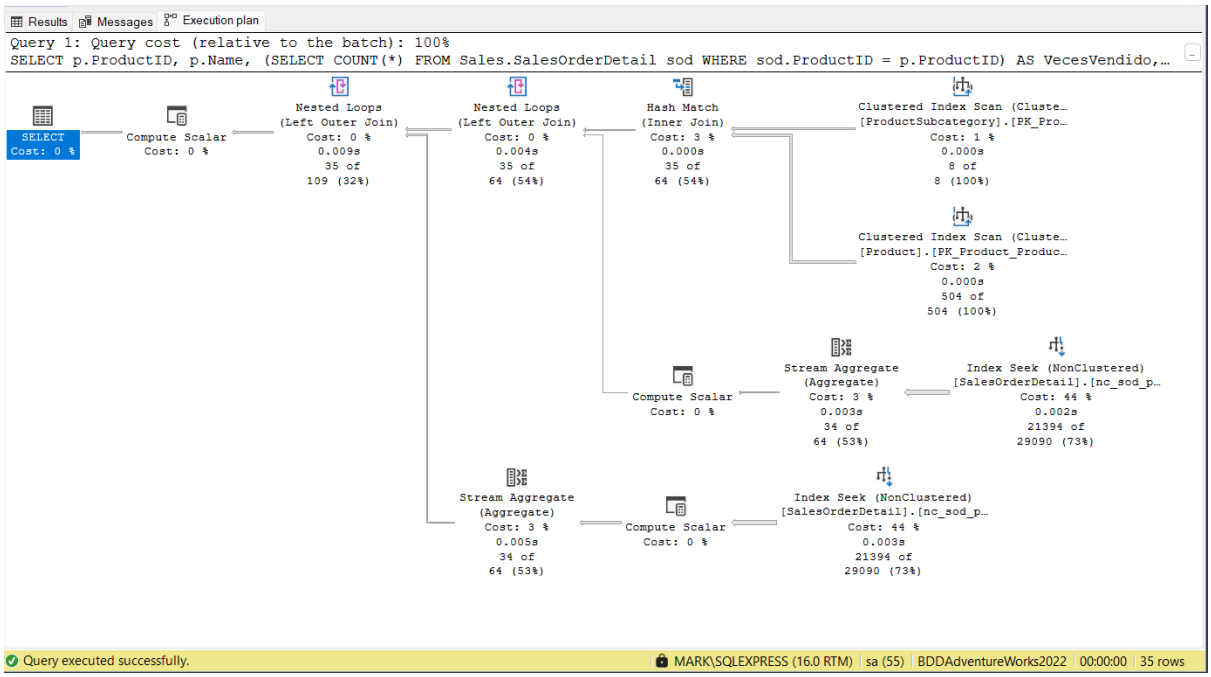


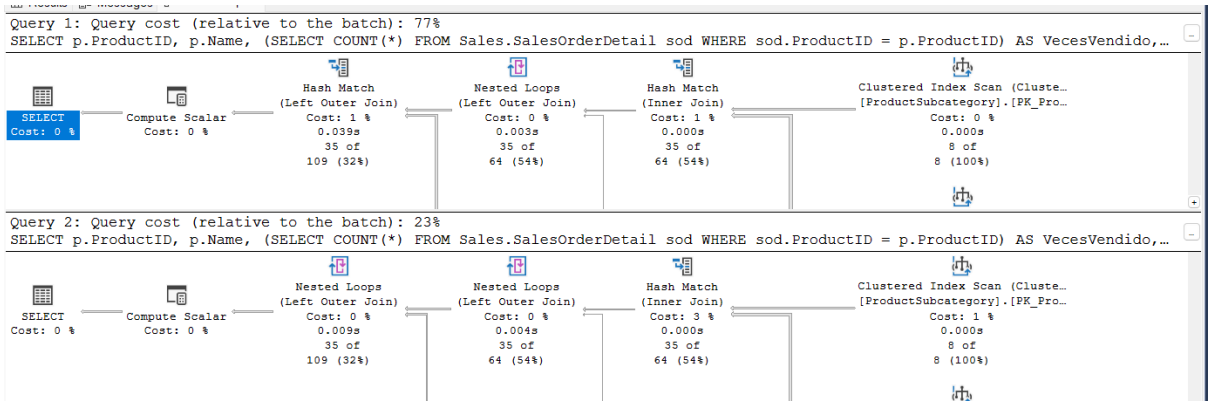
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	2026-03-22	81 ms	47 ms	SalesOrderDetail 1468 Product 15 ProductSubcategory 2 Worktable Workfile	53 Clustered Index Scan 30 Hash Match para obtener los ingresos por producto	La subconsulta para obtener los ingresos por producto es muy costosa	Crear un índice no agrupado para esa subconsulta.

Como se esperaba, la creación de este índice provocó un Index Seek y un Stream Agregate en lugar de un Clustered Index Scan con un Hash Match, reduciendo el costo del query a

23%, comparándolo con el original que tiene un costo de 77%. De igual manera las lecturas lógicas disminuyeron al costo de aumentar el número de exámenes en SalesOrderDetail.

```
create nonclustered index nc_sod_productid
on Sales.SalesOrderDetail(ProductID) include (OrderQty, UnitPrice)
```

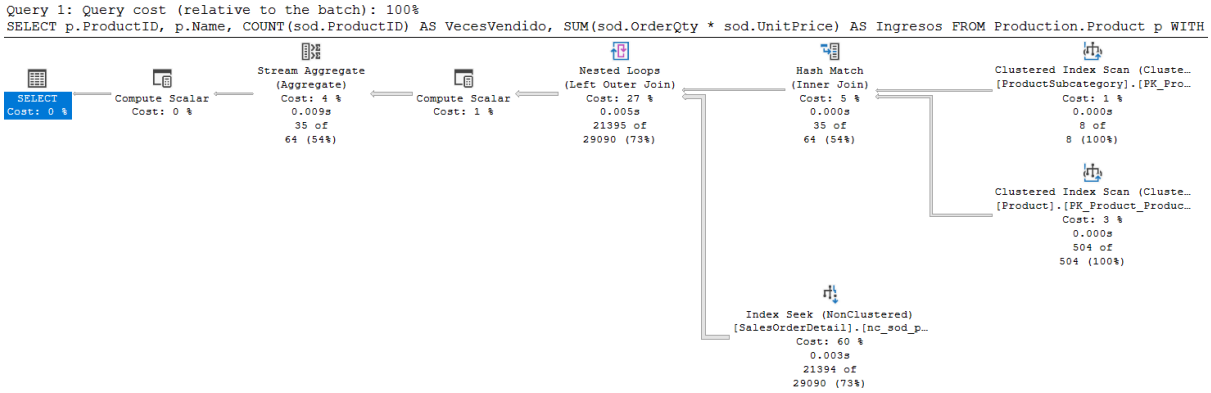




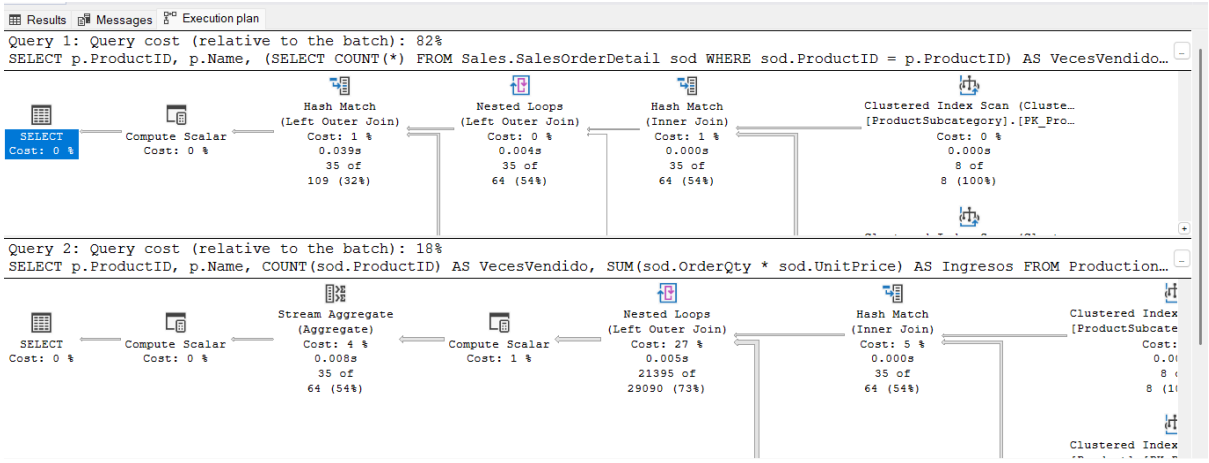
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	2026-03-22	43 ms	16 ms	SalesOrderDetail 496 con 70 exámenes Product 15 ProductSubcategory 2 Worktable Workfile	44 Index Seek para obtener los ingresos por producto 44 Index Seek para contar las veces que un producto se ha vendido	El plan de ejecución sigue siendo muy complejo por hacer las subconsultas correlacionadas.	Reescribir la consulta

Reescribir la consulta

```
SELECT p.ProductID, p.Name,
       COUNT(sod.ProductID) AS VecesVendido,
       SUM(sod.OrderQty * sod.UnitPrice) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID =
psc.ProductSubcategoryID
LEFT JOIN Sales.SalesOrderDetail sod ON sod.ProductID = p.ProductID
WHERE psc.ProductCategoryID = 3
GROUP BY p.ProductID, p.Name;
```



Al reescribir la consulta, el plan de ejecución se simplificó bastante, ya no hay tantos flujos de procesamiento y se puede leer de una forma mucho más clara. De la misma manera se redujeron las lecturas lógicas, aunque el tiempo de procesamiento quedó relativamente igual. Cabe notar que la consulta original es 4 veces más pesada para el equipo que la consulta actual.

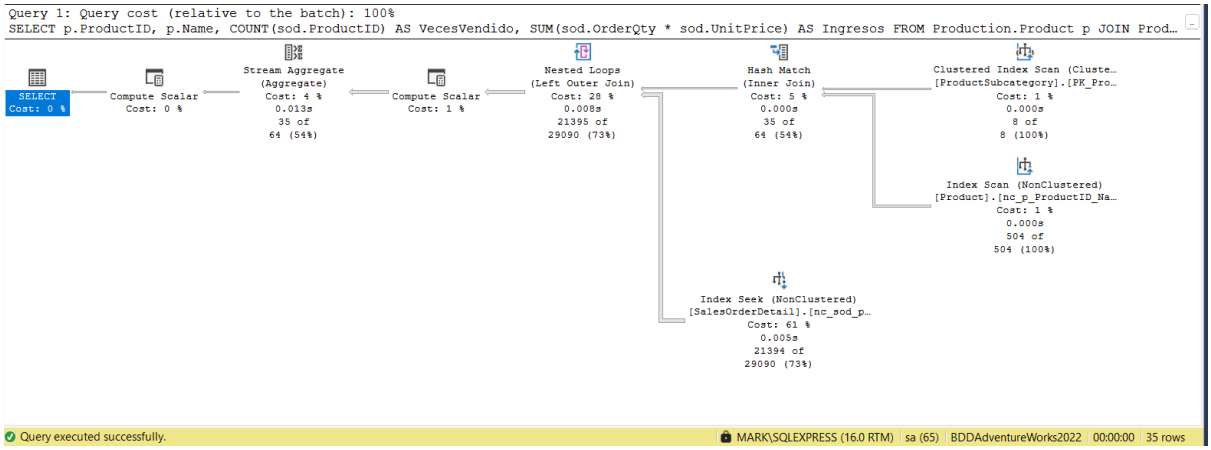


Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	2026-03-22	47 ms	15 ms	SalesOrderDetail 248 con 35 exámenes Product 15 ProductSub	60 Index Seek para obtener datos de SalesOrderDetail	Se realiza un Nested Loops que pasa de 30,000 a 21,395 filas	Con esta reescritura de la consulta, se puede proponer otro índice sobre product con una llave compuesta con los parámetros del

				category 2 Workfile Worktable			Group By y con la condición de reunión. El crear índices para buscar en las tablas Product y ProductSubcategory quitaría los workfile y wortable pero no bajan tanto las lecturas lógicas y tampoco aportan mucho al procesamiento.

Después de la creación del índice, las lecturas lógicas se reducen bastante comparado a los otros.

```
create nonclustered index nc_p_ProductID_Name
on Production.Product(ProductID, Name) include (ProductSubcategoryID)
```



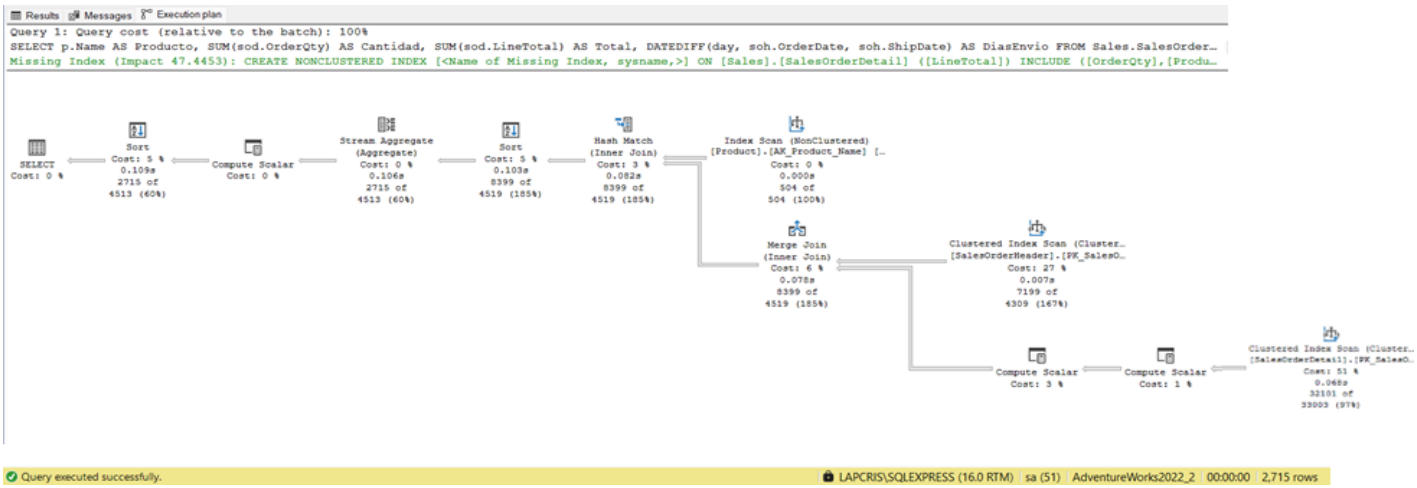

```
(35 rows affected)
Tabla "SalesOrderDetail". Número de examen 35, lecturas lógicas 248, lecturas físicas 0, lectur
Tabla "Workfile". Número de examen 0, lecturas lógicas 0, lecturas físicas 0, lecturas de servi
Tabla "Worktable". Número de examen 0, lecturas lógicas 0, lecturas físicas 0, lecturas de serv
Tabla "Product". Número de examen 1, lecturas lógicas 6, lecturas físicas 0, lecturas de servid
Tabla "ProductSubcategory". Número de examen 1, lecturas lógicas 2, lecturas físicas 0, lectura

(1 row affected)

 Tiempos de ejecución de SQL Server:
   Tiempo de CPU = 16 ms, tiempo transcurrido = 94 ms.
Tiempo de análisis y compilación de SQL Server:
   Tiempo de CPU = 0 ms, tiempo transcurrido = 0 ms.

 Tiempos de ejecución de SQL Server:
   Tiempo de CPU = 0 ms, tiempo transcurrido = 0 ms.

Completion time: 2026-03-26T22:03:47.1133879-06:00
```

CONSULTA 10 (Cristobal Olvera Valdivia)

```sql
-- 10
SELECT p.Name AS Producto,
       SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total,
       DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
  AND DATEPART(quarter, soh.OrderDate) = 2
  AND sod.LineTotal > 1000
GROUP BY  p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;
```

- **Análisis de la consulta original:**

La consulta original tiene como objetivo obtener las ventas agrupadas por producto, calculando cantidades, totales y los días de envío de aquellos pedidos del segundo trimestre con un total mayor a 1000. Sin embargo, se detectó la presencia de un JOIN innecesario a la tabla Sales.Customer, ya que no se extrae ninguna columna de ella ni se usa como filtro.



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones
10	21/03/2026	379 ms	141 ms	SalesOrderDetail: 1,238 SalesOrderHeader: 665	Clustered Index Scan (51% y 27%)	1. Alto costo de I/O por Clustered Index Scans. 2. Uso de funciones en el WHERE (DATEDIFF, DATEPART) que generaban consultas no sargables, impidiendo búsquedas directas. 3. JOIN innecesario (Sales.Customer)	1. Crear índices de cobertura no agrupados para ambas tablas. 2. Eliminar JOIN sin uso. 3. Reescribir la lógica de la consulta para usar rangos SARGables.

- **Propuesta de solución:**

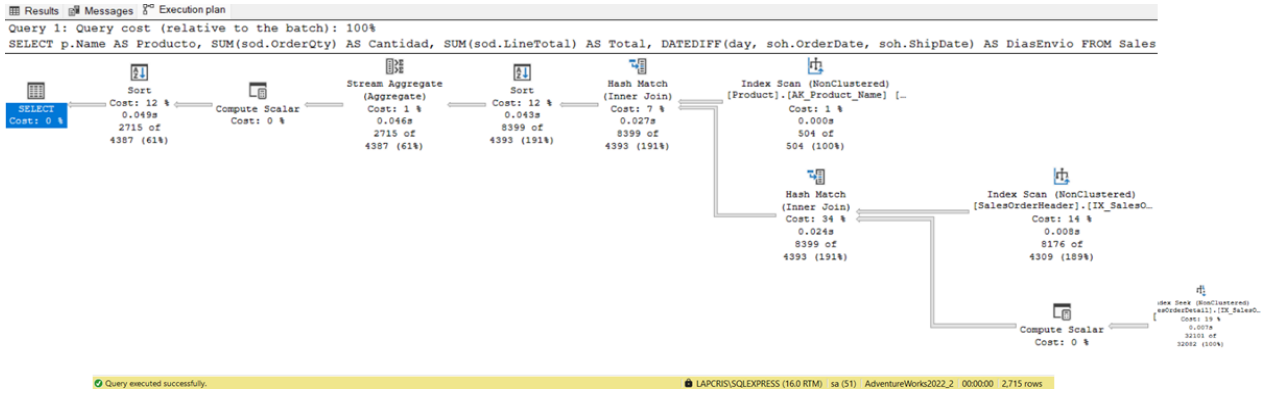
1. Primera modificación: Creación de índices de cobertura (Non-Clustered Indexes) El primer problema a resolver son las lecturas excesivas. Para evitar que SQL Server lea toda la estructura de la tabla original (Clustered Index Scan), se crearon dos índices "de cobertura" delgados.

- Para SalesOrderDetail, se indexó la columna de búsqueda LineTotal y se incluyeron (INCLUDE) las columnas OrderQty y ProductID.

- Para SalesOrderHeader, se indexó OrderDate y se incluyó ShipDate. Al usar la cláusula INCLUDE, nos aseguramos de que el motor encuentre todos los datos que la consulta necesita (el SELECT y el JOIN) directamente en el índice, evitando el costoso salto a la tabla original conocido como *Key Lookup*

```sql
create nonclustered index IX_SalesOrderDetail_LineTotal_Optimizacion
on Sales.SalesOrderDetail (LineTotal)
include (OrderQty, ProductID);

create nonclustered index IX_SalesOrderHeader_Fechas_Optimizacion
on Sales.SalesOrderHeader (OrderDate)
include (ShipDate);
```

```sql
SELECT p.Name AS Producto,
       SUM(sod.OrderQty) AS Cantidad,
       SUM(sod.LineTotal) AS Total,
       DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
  AND DATEPART(quarter, soh.OrderDate) = 2
  AND sod.LineTotal > 1000
GROUP BY  p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;
```

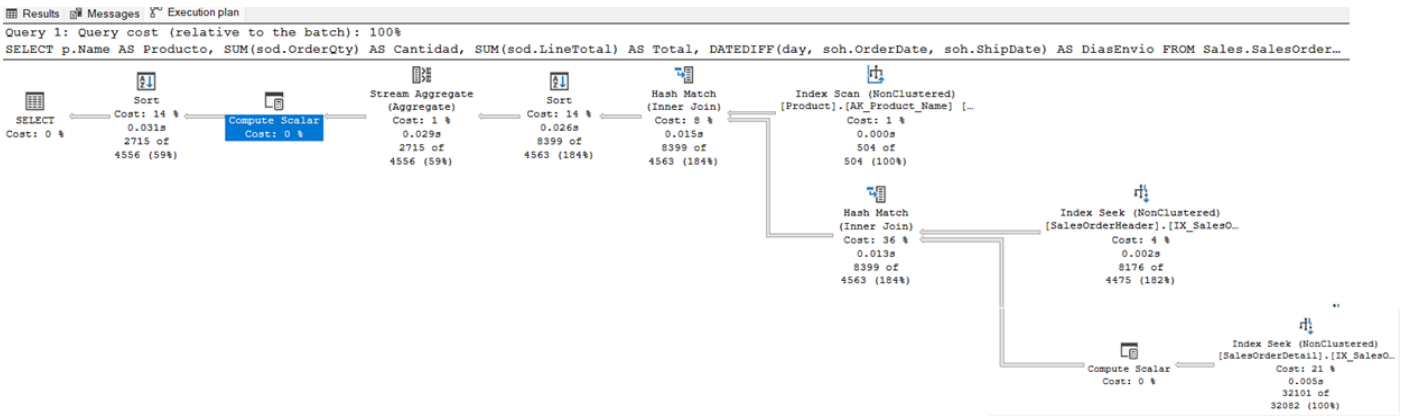


Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones
10	21/03/2026	127 ms	16 ms	SalesOrderDetail: 152 SalesOrderHeader: 104	Index Seek (19%) Index Scan (14%)	Se logró reducir el I/O, pero el uso de la función DATEPART obliga a realizar un Index Scan en la tabla de cabeceras.	Reescribir la consulta convirtiéndola en SARGable para lograr un Index Seek completo.

2. Segunda modificación: Reescritura de la consulta (Consultas SARGables)
Incluso con los índices creados, la condición original DATEPART(quarter, soh.OrderDate) = 2 impedía que el motor usará el nuevo índice de fechas eficientemente. Al usar una función matemática sobre la columna, SQL Server se ve forzado a escanear todo el índice. Para solucionar esto, se eliminó la función y se reescribió el código utilizando rangos de fechas explícitas con operadores >= y < para cada año registrado en la base de datos (2011 al 2014). Esto convierte a la consulta en **SARGable** (Search Argument Able), permitiendo que el motor salte directamente a los datos requeridos mediante un *Index Seek*. Adicionalmente, se eliminó el JOIN a la tabla Customer.

```sql
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
  AND (
        (soh.OrderDate >= '20110401' AND soh.OrderDate < '20110701') OR
        (soh.OrderDate >= '20120401' AND soh.OrderDate < '20120701') OR
        (soh.OrderDate >= '20130401' AND soh.OrderDate < '20130701') OR
        (soh.OrderDate >= '20140401' AND soh.OrderDate < '20140701')
      )
```

```sql
SELECT p.Name AS Producto,
       SUM(sod.OrderQty) AS Cantidad,
       SUM(sod.LineTotal) AS Total,
       DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
  AND (
        (soh.OrderDate >= '20110401' AND soh.OrderDate < '20110701') OR
        (soh.OrderDate >= '20120401' AND soh.OrderDate < '20120701') OR
        (soh.OrderDate >= '20130401' AND soh.OrderDate < '20130701') OR
        (soh.OrderDate >= '20140401' AND soh.OrderDate < '20140701')
      )
  AND sod.LineTotal > 1000
GROUP BY  p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;
```



Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan de Costo (%)	Issues Identificados	Recomendaciones
10	21/03/2026	108 ms	31ms	SalesOrderDetail: 152 SalesOrderHeader: 38	Index Seek en SOH (4%) Index Seek en SOD (21%)	Optimización extrema lograda combinando estructuras físicas y reescritura de código.	Mantener índices de cobertura y estructura SARGable.